

Data Handling: Import, Cleaning and Visualisation

Sample Solution for Exercise/Workshop 3: Programming with Data/HTML Web Scraping

Aurélien Sallin (lecturer) and Matthias Roesti (class tutor)

25/10/2023

Exercises

Exercise A: Write a Sum Function

In the lecture on Programming with Data, we have discussed the most basic programming concepts in R. Among other things, you have learned how we can use for-loops to sum up numbers stored in a numeric vector.

1. Use the code example in the respective lecture notes and the following empty function construct to implement a function that takes a numeric vector as input and returns the sum of the vector's elements.

```
my_sum <- function(x){  
  
  # Initialize the total - we start at 0  
  total <- 0  
  
  for (num in x){  
    total <- total + num  
  }  
  
  return(total)  
}
```

2. Define the vector `numbers = c(1, 2, 3)`. Then, compare your function `my_sum()` with the `sum()` function (the pre-implemented sum function in R) when using `numbers` as the input argument: Do you get the same output for `my_sum(numbers)` and `sum(numbers)`?

We get the same result in both cases - our function seems to be working:

```
numbers = c(1,2,3)
```

```
my_sum(numbers)
```

```
## [1] 6
```

```
sum(numbers)
```

```
## [1] 6
```

Exercise B: Robustness and Warnings

1. Once you have implemented and successfully tested `my_sum()`, call your function with the following vector as input. What happens?

```
numbers2 <- c("1", "2", "3")
```

In this case We get an error, R will not compute the sum and complains about a non-numeric argument: 'Error in total + num : non-numeric argument to binary operator'

```
my_sum(numbers2)
```

2. Why do we get an error? Investigate by comparing the vector `numbers` from Exercise A with the new vector `numbers2`. Hint: the functions `str()` and `class()` might help you.

We can see that the numbers2 vector we provided does not contain numeric values but strings, i.e. a series of characters:

```
# Investigate the structures
```

```
str(numbers)
```

```
## num [1:3] 1 2 3
```

```
str(numbers2)
```

```
## chr [1:3] "1" "2" "3"
```

```
# Check the classes
```

```
class_numbers <- class(numbers)
```

```
class_numbers2 <- class(numbers2)
```

```
print(paste("Class of 'numbers':", class_numbers))
```

```
## [1] "Class of 'numbers': numeric"
```

```
print(paste("Class of 'numbers2':", class_numbers2))
```

```
## [1] "Class of 'numbers2': character"
```

3. Extend your `my_sum()` function with a control statement that checks whether the input is of the right class for the function to work (Hint: `class(x)!="numeric"`). The goal is to make the function more robust to wrong input. That is, if the input is wrong, issue a warning but don't break down with an error. Use the following code for the warning message:

```
warning("Wrong input! This function only accepts numeric or integer values!")
```

We can fix our code by adding an if-statement to make sure we get a warning in case the input is not numeric. The `is.numeric()` command will return `TRUE` if `x` is numeric (and also if `x` is an integer, hence it is more flexible than requiring `class == "numeric"` for example)

```
my_sum <- function(x){
  total <- 0

  # Check for the right input class
  if (is.numeric(x)) {
    for (num in x){
      total <- total + num
    }
  } else {
    warning("Wrong input! This function only accepts numeric or integer values!")
    total <- NA_real_ # if no sum can be computed, we return a missing value
  }

  return(total)
}
```

Note that we would not want to use our own function to sum stuff up, because on top of not being able to address missing entries (easy to fix, see first exercise), it is a lot slower:

```
# Creating vector with 100 million random (normally distributed) numbers
big_vector = rnorm(100000000)
# Time the built-in sum function
time_builtin_sum <- system.time({
  sum(big_vector)
})[3] # Extracting the elapsed time

# Time the custom my_sum function
time_my_sum <- system.time({
  my_sum(big_vector)
})[3] # Extracting the elapsed time

# Print the results
print(paste("Execution time of built-in sum:", time_builtin_sum, "seconds"))
```

```
## [1] "Execution time of built-in sum: 0.0800000000000001 seconds"
```

```
print(paste("Execution time of custom my_sum:", time_my_sum, "seconds"))
```

```
## [1] "Execution time of custom my_sum: 2.14 seconds"
```

4. Test your enhanced sum function with the two input vectors `numbers` and `numbers2`.

We can see that we get a warning now when running the function on the second vector. Importantly, the function also returns a missing value for the sum, such that we are aware that no valid result is available. Specifically, the function should not return the value of 0 we initially set for the `total` in the function above, because that may cause a lot of issues if interpreted as a proper result.

```
my_sum(numbers)
```

```
## [1] 6
```

```
my_sum(numbers2)
```

```
## Warning in my_sum(numbers2): Wrong input! This function only accepts numeric or integer values!
```

```
## [1] NA
```

Exercise C: Web Scraping Exercise (advanced): Importing Data from a HTML Table (on a website)

Previously, we discussed the *Hypertext Markup Language (HTML)* as code to define the structure/content of a website and HTML-documents as semi-structured data sources. The following exercise contains the basic steps necessary to import data from an HTML table into R.

The aim of the exercise is to generate a CSV file containing data on ‘divided government’ in US politics. We use the following Wikipedia page as a data source: https://en.wikipedia.org/wiki/Divided_government_in_the_United_States. The page contains a table indicating the president’s party, and the majority party in the US House and the US Senate per Congress (2-year periods). The first few rows of the cleaned data are supposed to look like this:

```
##   year  president senate house
## 1: 1861    Lincoln      R      R
## 2: 1862    Lincoln      R      R
## 3: 1863    Lincoln      R      R
## 4: 1864    Lincoln      R      R
```

```
## 5: 1865 A. Johnson      R      R
## 6: 1866 A. Johnson      R      R
```

In a first step, we initiate fix variables for paths and load additional R packages needed to handle data stored in HTML documents. NOTE: the `OUTPUT_PATH` specified here assumed that there is a folder called `data` in the current working directory (get your current working directory by running `getwd()`) - adjust it if you that is not the case or if you want to store the output somewhere else.

```
# SETUP -----

# load packages
library(rvest)
library(data.table)

# fix vars
SOURCE_PATH <- "https://en.wikipedia.org/wiki/Divided_government_in_the_United_States"
OUTPUT_PATH <- "data/divided_gov.csv"
```

Now we write the part of the script that fetches the data from the Web. This part consists of three steps.

1. Fetch the entire website (HTML document) from Wikipedia with (`read_html()`).

```
# FETCH/FORMAT DATA -----

# fetch from web
doc <- read_html(SOURCE_PATH)
```

2. Extract the part of the website containing the table with the data we want (hint: use `html_table()` on the HTML document you just fetched, and see if one of the resulting tables in the list generated by this function matches what we are looking for, you also want the option `fill=TRUE` inside that function).

```
tab <- html_table(doc, fill=TRUE)[[2]] # select the second ta
```

3. Finally, check that the table has been properly parsed (you may want to remove the last line if it contains code junk) and store its content in a data frame called `tab`.

When inspecting the table, we see that the last row also contains the column names - will remove this (note that the minus sign can be used for a negative selection, i.e. to select rows that you do not want, and `nrow` will give us the index of the last, n-th row):

```
tab <- tab[-nrow(tab), ] # remove last row (not containing data)
```

4. Clean the data to get a data set more suitable for data analysis. Note that the original table contains information per congress (2-year periods). However, as the sample above shows, we aim for a panel at the year level, i.e. there should be a separate observation for every individual year. The following code iterates through the rows of the data frame and generates for each row per congress several two rows (one for each year in the congress).¹ Try to understand what each line of the following code does (it should run as is if the data frame `tab` had the right format):

Now things get a bit more complicated - see my comments in the code:

```
all_years <- list() # creating empty list object to
n <- length(tab$Year) # determining the number of years we have to iterate over
length(all_years) <- n # pre-allocating space in the list (not strictly necessary,
# but can improve speed/memory efficiency in some cases)

for (i in 1:n){
```

¹See `?strsplit`, `?unlist`, and this introduction to regular expressions for the background of how this is done in the code example here.

```

row <- tab[i,] # choosing row number i of the input table
y <- row$Year # extracting current year

# In the following line, we extract the beginning year,
# for example if the year column says '2005-2007', begin would take the value '2005'
# and end would take the value '2007'.
# This is done in three steps:
# - first, strsplit() creates a list with two elements based on a regular
#   expression "[\\-\\-]", which indicates that the split should be before
#   and after either a '-' or '-' character. The option 'perl = TRUE' enables
#   use of Perl-compatible (read: more intricate/fancy) regular expressions.
#   To understand the elements of a regular expression, ChatGPT or tools such as
#   'regexr.com' can be useful
# - second, the 'unlist()' command converts the list of elements generated by
#   strsplit() into a vector, and we choose the first element for the 'begin'
#   part, and the second element (end year) for the 'end' part
# - third, we convert the year character string into a numeric value with
#   'as.numeric()'
begin <- as.numeric(unlist(strsplit(x = y, split = "[\\-\\-]", perl = TRUE))[1])
end <- as.numeric(unlist(strsplit(x = y, split = "[\\-\\-]", perl = TRUE))[2])

# with these two values, we then create a new data frame where we have all the
# years from begin to end (not including end) in the first column, and the parties
# of president, senate and house majority in columns 2-4. Note that we exclude the
# end year because congress is handed over in January of the final year, meaning the
# final year is actually already the year the next congress is in power.
# In addition, note that r will recycle the single value provided in columns 2-4
# as often as needed to fill the number of rows of the data frame. R does this as
# long as the longest column vector is an exact multiple of the shorter vectors -
# KEEP IN MIND, AS THIS CAN LEAD TO UNDESIRED BEHAVIOR IF YOU DO NOT WANT THIS TO
# HAPPEN
tabnew <- data.frame(year=begin:(end-1), president=row$President, senate=row$Senate,
                     house=row$House)

all_years[[i]] <- tabnew # adding the newly create observations to the list
}

allyears <- bind_rows(all_years) # combining our list of observations into a data frame

```

5. In a last step, inspect the collected data and write it to a CSV file, using the OUTPUT_PATH defined at the beginning of this exercise to specify where the file should be stored.

```

# WRITE TO DISK -----

# inspect
head(allyears)

# note: with head, you may not always get the best impression, and with a small dataframe
# such as this one, viewing the entire one may be a better option to spot any pot. issues:
view(allyears)

# remove unwanted Wikipedia footnote references '[a]' etc., note again the use of a regular
# expression to do so
allyears$senate <- str_remove_all(allyears$senate, "\\[[^\\]]*\\]")

```

```

allyears$house <- str_remove_all(allyears$house, "\\[[^\\]]*\\]")

# write to CSV
write_csv(allyears, file=OUTPUT_PATH)

```

Additional Exercises for Training (not exam-relevant)

Exercise D: Standard Deviation Function

1. During the lecture on Programming with Data we have step-by-step implemented our own R function to compute the mean. Now implement an R function called `my_sd` that computes the standard deviation, defined as: $SD = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$. Hint: revisit the math operators section in the notes on the first exercise session.

```

my_sd <- function(x) {
  n <- length(x)

  # First, check if n is greater than 1
  if(n <= 1) {
    warning("At least two numbers are required to compute standard deviation.")
    return(NA) # NA is returned if computation is not possible
  }

  mean_x <- mean(x)
  sum_squared_diff <- sum((x - mean_x)^2)
  sd <- sqrt(sum_squared_diff / (n - 1))

  return(sd)
}

```

2. Test your implementation by comparing its output with the already implemented `sd()` function.

```

# Create a test vector
set.seed(2023) # setting seed to reproduce the same results (note that results may still
               # vary across different devices and R-versions)
test_vector <- rnorm(10000) # a random vector of 10000 numbers with mean zero and sd of 1

# Compare the outputs of the custom function and the built-in function
custom_sd <- my_sd(test_vector)
builtin_sd <- sd(test_vector)

print(paste("Custom standard deviation:", custom_sd))

## [1] "Custom standard deviation: 0.996152686303493"
print(paste("Built-in standard deviation:", builtin_sd))

## [1] "Built-in standard deviation: 0.996152686303493"

```

Exercise E: Standard Error Function

Building on the function written in Exercise D, implement a function to compute (the estimate of) the sample mean standard error as defined here: $SE_{\bar{x}} = \frac{SD}{\sqrt{n}}$. Hint: the idea is to use your own implementation (in Exercise D) to compute SD as a first step of your standard error function.

Exercises D and E should illustrate how you can approach more complex problems by breaking them up in smaller problems and addressing each of these step-by-step. That is, when looking at the problem of computing $SE_{\bar{x}} = \frac{SD}{\sqrt{n}}$, you see that computing SD is a part of the overall problem, which can be addressed separately.

```
my_se <- function(x) {
  # Call the previously defined my_sd function to compute the standard deviation
  sd <- my_sd(x)

  n <- length(x)
  if(n <= 1) {
    warning("At least two numbers are required to compute standard error.")
    return(NA) # NA is returned if computation is not possible
  }

  se <- sd / sqrt(n)
  return(se)
}

# Test the function (base R has not built in function for this - we will test the
# final output below)
test_se <- my_se(test_vector)
print(paste("Standard error computed by my function:", test_se))

## [1] "Standard error computed by my function: 0.00996152686303493"
```

Exercise F: T-test

Implement a function for the one-sample t-test: $t = \frac{\bar{x} - \mu_0}{SE}$.

```
my_t_test <- function(x, mu_0) {
  # Calculate the sample mean
  sample_mean <- mean(x)

  # Compute standard error using the function from Exercise E
  se <- my_se(x)

  n <- length(x)
  if(n <= 1) {
    warning("At least two numbers are required to compute the t-statistic.")
    return(NA) # NA is returned if computation is not possible
  }

  # Calculate the t-statistic
  t_statistic <- (sample_mean - mu_0) / se

  return(t_statistic)
}

# Testing against R

# Test the function with a sample vector and a test mean (mu_0)
mu_0_test <- 0 # This is the hypothesized mean (and the true mean of our data)
```

```

# Because our hypothesized value is correct, we should not reject, i.e.
# get a low t-statistic, well below the 5% significance threshold of +-1.96
t_test_result <- my_t_test(test_vector, mu_0_test)
print(paste("T-test result:", t_test_result))

## [1] "T-test result: -0.781507395524073"

# Now, use the built-in t-test function for comparison
builtin_t_test_result <- t.test(test_vector, mu = mu_0_test)$statistic

print(paste("Built-in T-test result:", builtin_t_test_result))

## [1] "Built-in T-test result: -0.781507395524073"

```

Old Exam Questions

Q1

“What is the dimension of the tibble that you get with the code below?”

```
read_csv("1,2,\n3,4", col_names = c("a", "b", "c"))
```

Q2 (one correct)

Given the vector `numbers` below, which statement will print -17?

```
numbers <- c(-14, 11, 800, -17, 5)
```

`print(numbers[4])`, `print(numbers, 4)`, `print(numbers[-17])`, or `print(numbers[3])`?

Q3 (one correct)

“Have a look at the following line of R code:

```
a <- data.frame(numbers=1:3, letters=c("y", "z", "x"))
a$years <- 2017:2019
```

Which of the following statements is correct (after all lines of code have been executed)?

1. The R code will result in an error message when executed.
2. The code initiates a `data.frame`-object containing three columns, one for each value in `letters`.
3. The code initiates a `data.frame`-object containing four rows.
4. The data-frame `a` contains three columns.

Q4 (one correct)

“Consider the following while-loop.

```
while(8>7) {print("TRUE")}
```

Which of the following statements about this while loop is correct?

1. The loop will print “TRUE” to the R console 8 times.
2. The loop will print “TRUE” to the R console 7 times.
3. The loop’s syntax is wrong. Once this loop is executed, an error message will be returned.
4. Once this loop is executed, it will go on forever (unless we shut down R).

Q5 (one correct)

“Consider the following function. Which statement below is correct?

```
my_function <- function(x) {  
  s = 0  
  n = 0  
  for (i in x) {  
    s = s + i  
    n = n + 1  
  }  
  result = s/n  
  return(result)  
}
```

1. We would get the same result for `result` if we replaced the line `result = s/n` by `result = s/length(x)`.
2. This piece of code will result in an error.
3. The output of this function would always be the same irrespective of whether we write `return(s)` or `return(result)`.
4. Running `my_function(c("1", "3", "2"))` would give us 2 as a result.

Q6 (multiple choice with 1+ correct)

“Consider the following code:

```
success_story <- c(  
  "WoW! This exam is soooo easy!",  
  "Aw!!! this hurts ",  
  "OMG!who would have thought"  
)  
  
split <- strsplit(success_story, split = "!", perl = TRUE)
```

(Hint: `length(x)` gets the length of `x`, and `lengths(x)` gets the length of each element of `x`.)

1. `lengths(split) = c(3,4,3)`
2. `lengths(split) = c(2,4,2)`
3. `length(split) = 3`
4. `lengths(split) = c(2,2,2)`